

Rashtreeya Sikshana Samithi Trust

RV Institute of Technology and Management®

(Affiliated to VTU, Belagavi)

JP Nagar, Bengaluru - 560076

Department of Computer Science and Engineering



Course Name: Cryptography

Course Code: 18CS744

VII Semester

2018 Scheme

Prepared By :

Dr. Deepak N A

Associate Professor,

Department of Computer Science and Engineering

RVITM, Bengaluru – 560076

Email: deepakna.rvitm@rvei.edu.in

MODULE 4

X.509 CERTIFICATES

4.1 Certificates

- X.509 is part of the X.500 series of recommendations that define a directory service, being a server or distributed set of servers that maintains a database of information about users.
- X.509 defines a framework for the provision of authentication services by the X.500 directory to its users. The directory may serve as a repository of public-key certificates
- Each certificate contains the public key of a user and is signed with the private key of a trusted certification authority. In addition, X.509 defines alternative authentication protocols based on the use of public-key certificates. X.509 is based on the use of public-key cryptography and digital signatures.
- The X.509 certificate format Fig 4.1 is widely used, in for example S/MIME, IP Security and SSL/TLS and SET. X.509 was initially issued in 1988. The standard was subsequently revised to address some of the security concerns; a revised recommendation was issued in 1993. A third version was issued in 1995 and revised in 2000.

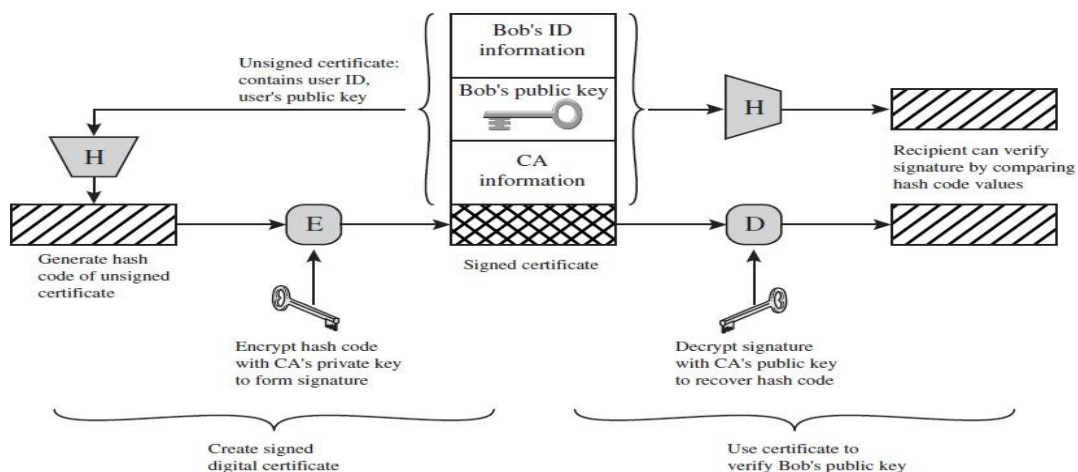


Fig 4.1: X.509Certificates

The heart of the X.509 scheme is the public-key certificate associated with each user. These user certificates are assumed to be created by some trusted certification authority (CA) and placed in the directory by the CA or by the user. The directory server itself is not responsible for the creation of public keys or for the certification function; it merely provides an easily accessible location for users to obtain certificates.

The standard uses the notation for a certificate Fig 4.2 of:

$CA\langle\langle A \rangle\rangle$ where the CA signs the certificate for user A with its private key. In more detail $CA\langle\langle A \rangle\rangle = CA \{V, SN, AI, CA, UCA, A, UA, Ap, TA\}$.

If the corresponding public key is known to a user, then that user can verify that a certificate signed by the CA is valid.

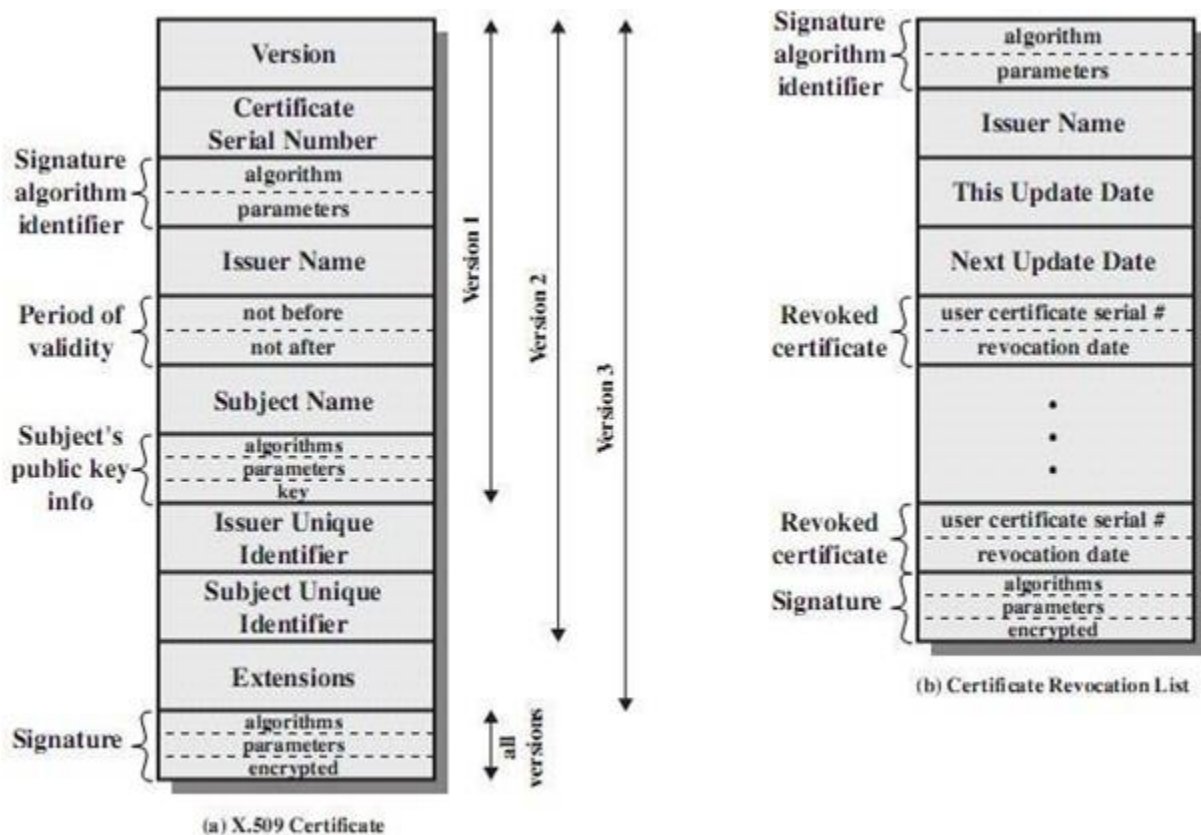


Fig 4.2: X.509 Formats

- [**Version:** Differentiates among successive versions of the certificate format; the default is version 1.
 - 1. If the issuer unique identifier or subject unique identifier are present, the value must be version 2.
 - 2. If one or more extensions are present, the version must be version 3.
- [**Serial number:** An integer value unique within the issuing CA that is unambiguously associated with this certificate.
- [**Signature algorithm identifier:** The algorithm used to sign the certificate together with any associated parameters. Because this information is repeated in the signature field at the end of the certificate, this field has little, if any, utility.
- [**Issuer name:** X.500 is the name of the CA that created and signed this certificate.
- [**Period of validity:** Consists of two dates: the first and last on which the certificate is valid.
- [**Subject name:** The name of the user to whom this certificate refers. That is, this certificate certifies the public key of the subject who holds the corresponding private key.
- [**Subject's public-key information:** The public key of the subject, plus an identifier of the algorithm for which this key is to be used, together with any associated parameters.
- [**Issuer unique identifier:** An optional-bit string field used to identify uniquely the issuing CA in the event the X.500 name has been reused for different entities.
- [**Subject unique identifier:** An optional-bit string field used to identify uniquely the subject in the event the X.500 name has been reused for different entities.
- [**Extensions:** A set of one or more extension fields. Extensions were added in version 3 and are discussed later in this section.
- [**Signature:** Covers all of the other fields of the certificate; it contains the hash code of the other fields encrypted with the CA's private key. This field includes the signature algorithm identifier.

The unique identifier fields were added in version 2 to handle the possible reuse of subject and/or issuer names over time. These fields are rarely used. The standard uses the following notation to define a certificate:

$$CA\langle\langle A \rangle\rangle = CA \{V, SN, AI, CA, UCA, A, UA, Ap, T^A\}$$

where

$Y\langle\langle X \rangle\rangle$ = the certificate of user X issued by certification authority Y

$Y \{I\}$ = the signing of I by Y. It consists of I with an encrypted hash

code appended

V = version of the certificate

SN = serial number of the certificate

AI = identifier of the algorithm used to sign the certificate

CA = name of certificate authority

UCA = optional unique identifier of the

CAA = name of user A

UA = optional unique identifier of the user

AAp = public key of user A

T^A = period of validity of the certificate

Obtaining a Certificate

User certificates generated by a CA have the following characteristics:

- Any user with access to the public key of the CA can verify the user public key that was certified.
- No party other than the certification authority can modify the certificate without this being detected.

Because certificates are unforgeable, they can be placed in a directory without the need for the directory to make special efforts to protect them.

If all users subscribe to the same CA, then there is a common trust of that CA. All user certificates can be placed in the directory for access by all users. In addition, a user can transmit his or her certificate directly to other users. In either case, once B is in possession of A's certificate, B has confidence that messages it encrypts with A's public key will be secure from eavesdropping and that messages signed with A's private key are unforgeable.

4.1.1 CA Hierarchy:

If both parties use the same CA, they know its public key and can verify others certificates. If there is a large community of users, it may not be practical for all users to subscribe to the same CA. Hence

there has to be some means to form a chain of certifications between the CA's used by the two parties, by the use of client and parent certificates. All these certificates of CAs by CAs need to appear in the directory, and the user needs to know how they are linked to follow a path to another user's public-key certificate. X.509 suggests that CAs be arranged in a hierarchy so that navigation is straightforward. It is assumed that each client trusts its parent's certificates.

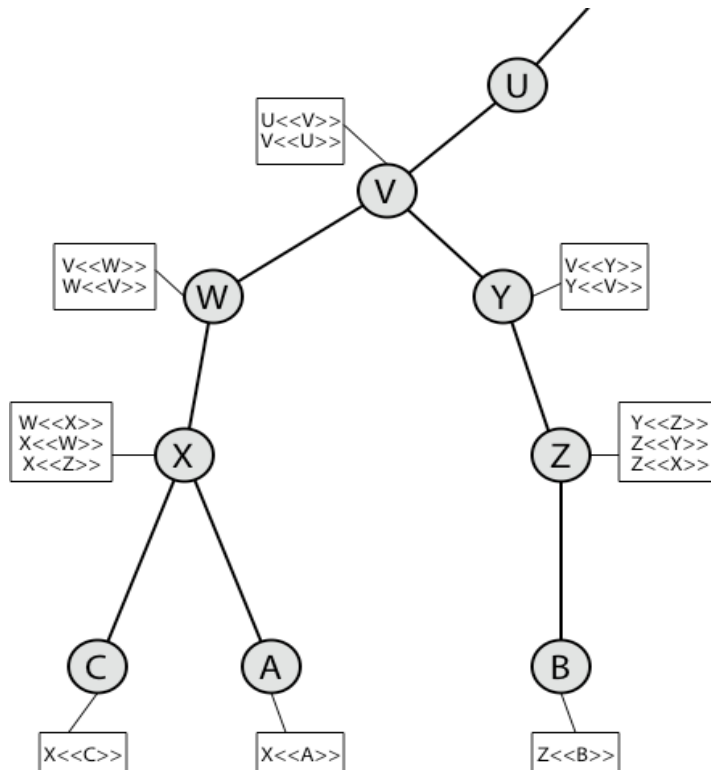


Fig 4.3: illustrates the use of an X.509 hierarchy

Above Fig 4.3 illustrates the use of an X.509 hierarchy to mutually verify clients certificates. The connected circles indicate the hierarchical relationship among the CAs; the associated boxes indicate certificates maintained in the directory for each CA entry. The directory entry for each CA includes two types of certificates:

Forward certificates: Certificates of X generated by other CAs,

Reverse certificates: Certificates generated by X that are the certificates of other CAs.

In this example, we can track chains of certificates as follows:

A acquires B certificate using chain
X<<W>>W<<V>>V<<Y>>Y<<Z>>Z<>
B acquires A certificate using chain:
Z<<Y>>Y<<V>>V<<W>>W<<X>>X<<A>>

4.1.2 Certificate Revocation:

A certificate includes a period of validity. Typically a new certificate is issued just before the expiration of the old one.

In addition, it may be desirable on occasion to revoke a certificate before it expires, for one of a range of following reasons:

1. The user's private key is assumed to be compromised.
2. The user is no longer certified by this CA. Reasons for this include that the subject's name has changed, the certificate is superseded, or the certificate was not issued in conformance with the CA's policies.
3. The CA's certificate is assumed to be compromised.

To support this, each CA must maintain a list consisting of all revoked but not expired certificates issued by that CA, known as the certificate revocation list (CRL). Each certificate revocation list (CRL) posted to the directory is signed by the issuer and includes the issuer's name, the date the list was created, the date the next CRL is scheduled to be issued, and an entry for each revoked certificate. Each entry consists of the serial number of a certificate and revocation date for that certificate. Because serial numbers are unique within a CA, the serial number is sufficient to identify the certificate.

When a user receives a certificate in a message, the user must determine whether the certificate has been revoked, by checking the directory CRL each time a certificate is received, this often does not happen in practice.

X.509 Version 3

The X.509 version 2 format does not convey all of the information. Rather than continue to add fields to a fixed format, standards developers felt that a more flexible approach was needed. X.509 version 3 includes a number of optional extensions that may be added to the version 2 format. Each extension consists of an extension identifier, a criticality indicator, and an extension value. The criticality indicator indicates whether an extension can be safely ignored or not.

4.1.3 Certificate Extensions

The certificate extensions fall into three main categories:

- **Key and policy information** - convey additional information about the subject and issuer keys, plus indicators of certificate policy. A certificate policy is a named set of rules that indicates the applicability of a certificate to a particular community and/or class of application with common security requirements.
- **Subject and issuer attributes** - support alternative names, in alternative formats, for a certificate subject or certificate issuer and can convey additional information about the certificate subject; eg. postal address, email address, or picture image
- **Certification path constraints** - allow constraint specifications to be included in certificates issued for CA's by other CA's that may restrict the types of certificates that can be issued by the subject CA or that may occur subsequently in a certification chain.

4.1.4 User Authentication

An authentication process consists of two steps:

- **Identification step:** Presenting an identifier to the security system. (Identifiers should be assigned carefully, because authenticated identities are the basis for other security services, such as access control service.)
- **Verification step:** Presenting or generating authentication information that corroborates the binding between the entity and the identifier.”

In essence, identification is the means by which a user provides a claimed identity to the system; user authentication is the means of establishing the validity of the claim. Note that user authentication is distinct from message authentication.

There are four general means of authenticating a user's identity, which can be used alone or in combination:

- **Something the individual knows:** Examples include a password, a personal identification number (PIN), or answers to a prearranged set of questions.
- **Something the individual possesses:** Examples include electronic keycards, smartcards, and physical keys. This type of authenticator is referred to as a *token*.
- **Something the individual is (static biometrics):** Examples include recognition by fingerprint, retina, and face.
- **Something the individual does (dynamic biometrics):** Examples include recognition by voice pattern, handwriting characteristics, and typing rhythm.

All of these methods, properly implemented and used, can provide secure user authentication. However, each method has problems. An adversary may be able to guess or steal a password. Similarly, an adversary may be able to forge or steal a token. A user may forget a password or lose a token. Further, there is a significant administrative overhead for managing password and token information on systems and securing such information on systems. With respect to biometric authenticators, there are a variety of problems, including dealing with false positives and false negatives, user acceptance, cost, and convenience.

Authentication Protocols

- An important application area is that of **mutual authentication** protocols. Such protocols enable communicating parties to satisfy themselves mutually about each other's identity and to exchange session keys. There, the focus was key distribution.
- Central to the problem of authenticated key exchange are two issues: **confidentiality and timeliness**.
- To prevent masquerade and to prevent compromise of session keys, essential identification and session key information must be communicated in encrypted form. The second issue, timeliness, is important because of the threat of message replays.

Replay Attacks: are where a valid signed message is copied and later resent. Such replays, at worst, could allow an opponent to compromise a session key or successfully impersonate another party. At minimum, a successful replay can disrupt operations by presenting parties with messages that appear genuine but are not.

4.1.5 Examples of replay attacks:

Simple replay: The opponent simply copies a message and replays it later.

Repetition that can be logged: An opponent can replay a time stamped message within the valid time window.

Repetition that cannot be detected: This situation could arise because the original message could have been suppressed and thus did not arrive at its destination; only there play

message arrives.

Backward replay without modification: This is a replay back to the message sender. This attack is possible if symmetric encryption is used and the sender cannot easily recognize the difference between messages sent and messages received on the basis of content.

Possible countermeasures include the use of:

- **Sequence numbers** (generally impractical since must remember last number used with every communicating party)
- **Timestamps** (needs synchronized clocks amongst all parties involved, which can be problematic)
- **Challenge/response** (using unique, random, unpredictable nonce, but not suitable for connectionless applications because of handshake overhead)

4.2 One-Way Authentication

One application for which encryption is growing in popularity is electronic mail (e-mail). The very nature of electronic mail, and its chief benefit, is that it is not necessary for the sender and receiver to be online at the same time. Instead, the e-mail message is forwarded to the receiver's electronic mailbox, where it is buffered until the receiver is available to read it. Accordingly, the e-mail message should be encrypted such that the mail-handling system is not in possession of the decryption key. A second requirement is that of authentication. Typically, the recipient wants some assurance that the message is from the alleged sender.

4.2.1 Remote User-Authentication Using Symmetric encryption

4.2.1.1 Mutual Authentication

As discussed earlier, A two-level hierarchy of symmetric encryption keys can be used to provide confidentiality for communication in a distributed environment. Usually involves the use of a trusted key distribution center (KDC). Each party in the network shares a secret master key with the KDC.

The KDC is responsible for generating session keys, and for distributing those keys to the parties involved, using the master keys to protect these session keys.

4.1.2.2 Needham-Schroeder Protocol

The Needham-Schroeder Protocol is the original, basic key exchange protocol. Used by 2 parties who both trusted a common key server, it gives one party the info needed to establish a session key with the other.

Note that all communications is between A&KDC and A&B, B&KDC don't talk directly (though indirectly a message passes from KDC via A to B, encrypted in B's key so that A is unable to read or alter it). Other variations of key distribution protocols can involve direct communications between B&KDC.

1. $A \rightarrow KDC: ID_A \parallel ID_B \parallel N_1$
2. $KDC \rightarrow A: E(K_a, [K_s \parallel ID_B \parallel N_1 \parallel E(K_b, [K_s \parallel ID_A])])$
3. $A \rightarrow B: E(K_b, [K_s \parallel ID_A])$
4. $B \rightarrow A: E(K_s, [N_2])$
5. $A \rightarrow B: E(K_s, [f(N_2)])$

Secret keys K_a and K_b are shared between A and the KDC and B and the KDC, respectively.

The purpose of the protocol is to distribute securely a session key K_s to A and B.

There is a critical flaw in the protocol, as shown. The message in step 3 can be decrypted, and hence understood only by B. But if an opponent, X, has been able to compromise an old session key, then X can impersonate A and trick B into using the old key by simply replaying step 3. Admittedly, this is a much more unlikely occurrence than

that an opponent has simply observed and recorded step 3.

Denning proposes to overcome this weakness by a modification to the Needham/Schroeder protocol that includes the addition of a timestamp to steps 2 and 3. Her proposal assumes that the master keys, K_a and K_b are secure, and it consists of the following steps.

1. $A \rightarrow KDC: ID_A || ID_B$
2. $KDC \rightarrow A: E(K_a, [K_s || ID_B || T || E(K_b, [K_s || ID_A || T])])$
3. $A \rightarrow B: E(K_b, [K_s || ID_A || T])$
4. $B \rightarrow A: E(K_s, N1)$
5. $A \rightarrow B: E(K_s, f(N1))$

T is a timestamp that assures A and B that the session key has only just been generated. Thus, both A and B know that the key distribution is a fresh exchange.

The Denning protocol seems to provide an increased degree of security compared to the Needham/Schroeder protocol. However, a new concern is raised: namely, that this new scheme requires reliance on clocks that are synchronized throughout the network. It points out a risk involved. The risk is based on the fact that the distributed clocks can become unsynchronized as a result of faults in the clocks or the synchronization mechanism.

The problem occurs **when a sender's clock is ahead of the intended recipient's clock.**

In this case, an opponent can intercept a message from the sender and replay it later when the timestamp in the message becomes current at the recipient's site. This replay could cause unexpected results. Gong refers to such attacks as **suppress-replay attacks.**

4.3 Remote User Authentication Using Asymmetric encryption

4.3.1 Mutual Authentication

This protocol assumes that each of the two parties is in possession of the current public key of the other. It may not be practical to require this assumption.

1. $A \rightarrow AS: ID_A \parallel ID_B$
2. $AS \rightarrow A: E(PR_{as}, [ID_A \parallel PU_a \parallel T]) \parallel E(PR_{as}, [ID_B \parallel PU_b \parallel T])$
3. $A \rightarrow B: E(PR_{as}, [ID_A \parallel PU_a \parallel T]) \parallel E(PR_{as}, [ID_B \parallel PU_b \parallel T]) \parallel E(PU_b, E(PR_a, [K_s \parallel T]))$

A protocol using timestamps is provided in that uses a central system, referred to as an authentication server (AS), because it is not actually responsible for secret key distribution. Rather, the AS provides public-key certificates. The session key is chosen and encrypted by A; hence, there is no risk of exposure by the AS. The timestamps protect against replays of compromised keys. See text for details. This protocol is compact but, as before, requires synchronization of clocks. Another approach, proposed by Woo and Lam, makes use of nonces.

1. $A \rightarrow KDC: ID_A \parallel ID_B$
2. $KDC \rightarrow A: E(PR_{auth}, [ID_B \parallel PU_b])$
3. $A \rightarrow B: E(PU_b, [N_a \parallel ID_A])$
4. $B \rightarrow KDC: ID_A \parallel ID_B \parallel E(PU_{auth}, N_a)$
5. $KDC \rightarrow B: E(PR_{auth}, [ID_A \parallel PU_a]) \parallel E(PU_b, E(PR_{auth}, [N_a \parallel K_s \parallel ID_B]))$
- 6. $B \rightarrow A: E(PU_a, [E(PR_{auth}, [(N_a \parallel K_s \parallel ID_B))] \parallel N_b])$
7. $A \rightarrow B: E(K_s, N_b)$

Note the authors themselves spotted a flaw in it and submitted a revised version of the algorithm in. In both this example and the protocols described earlier, protocols that appeared secure were revised after additional analysis. These examples highlight the difficulty of getting things right in the area of authentication.

4.3.2 One-Way Authentication

We have already presented public-key encryption approaches that are suited to electronic mail, including the straightforward encryption of the entire message for confidentiality, authentication, or both. These approaches require that either the sender know the recipient's public key (confidentiality) or the recipient know the sender's public key (authentication) or both (confidentiality plus authentication).

If confidentiality is the primary concern, then better to encrypt the message with a one-time secret key, and also encrypt this one-time key with B's public key. If authentication is the primary concern, then a digital signature may suffice (but could be replaced by an opponent). To counter such a scheme, both the message and signature can be encrypted with the recipient's public key. The latter two schemes require that B know A's public key and be convinced that it is timely. An effective way to provide this assurance is the digital certificate.

4.3 Kerberos (Important Topic)

Kerberos is an authentication service developed as part of Project Athena at MIT, and is one of the best known and most widely implemented **trusted third party** key distribution systems.

Kerberos provides a centralized authentication server whose function is to authenticate users to servers and servers to users. Unlike most other authentication schemes, Kerberos relies exclusively on symmetric encryption, making no use of public-

key encryption. Two versions of Kerberos are in common use: v4 & v5.

In a more open environment, in which network connections to other machines are supported, an approach that requires the user to prove his or her identity for each service invoked, and also require that servers prove their identity to clients, is needed to protect user information and resources housed at the server. Kerberos supports this approach, and assumes a distributed client/server architecture that employs one or more Kerberos servers to provide an authentication service. The first published report on Kerberos[STEI88] listed the following requirements:

- **Secure:** A network eavesdropper should not be able to obtain the necessary information to impersonate a user.
- **Reliable:** For all services that rely on Kerberos for access control, lack of availability of the Kerberos service means lack of availability of the supported services. Hence, Kerberos should be highly reliable and should employ a distributed server architecture, with one system able to back up another.
- **Transparent:** Ideally, the user should not be aware that authentication is taking place, beyond the requirement to enter a password.
- **Scalable:** The system should be capable of supporting large numbers of clients and servers. This suggests a modular, distributed architecture.

To support these requirements, Kerberos is a trusted third-party authentication service that uses a protocol based on that proposed by Needham and Schroeder which was discussed earlier in this chapter.

4.4 The Version 4 Authentication Dialogue

- Kerberos V4 is a basic third-party authentication scheme.
- The core of Kerberos is the **Authentication server (AS)** and **Ticket Granting Servers (TGS)** – these are trusted by all users and servers and must be securely administered.
- The protocol includes a sequence of interactions between the client, AS, TGT and desired server. Version 4 of Kerberos makes use of DES, in a rather elaborate
- protocol, to provide the authentication service

The heart of the first problem is the lifetime associated with the ticket-granting ticket. If this lifetime is very short (e.g., minutes), then the user will be repeatedly asked for a password. If the lifetime is long (e.g., hours), then an opponent has a greater opportunity for replay. Similarly, if an opponent captures a service-granting ticket and uses it before it expires, the opponent has access to the corresponding service.

The second problem is that there may be a requirement for servers to authenticate themselves to users.

First, consider the problem of captured ticket-granting tickets and the need to determine that the ticket presenter is the same as the client for whom the ticket was issued. An efficient way of doing this is to use a session encryption key to secure information. Table 4.1 shows the technique for distributing the session key.

Table 4.1: Summary of KEROBES

(1) $C \rightarrow AS \ ID_C \parallel ID_{TGS} \parallel TS_1$ (2) $AS \rightarrow C \ E(K_{C,TGS}, [K_{C,TGS} \parallel ID_{TGS} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{TGS}])$ $Ticket_{TGS} = E(K_{TGS}, [K_{C,TGS} \parallel ID_C \parallel AD_C \parallel ID_{TGS} \parallel TS_2 \parallel Lifetime_2])$ (a) Authentication Service Exchange to obtain ticket-granting ticket
(3) $C \rightarrow TGS \ ID_V \parallel Ticket_{TGS} \parallel Authenticator_c$ (4) $TGS \rightarrow C \ E(K_{C,TGS}, [K_{C,V} \parallel ID_V \parallel TS_4 \parallel Ticket_V])$ $Ticket_{TGS} = E(K_{TGS}, [K_{C,TGS} \parallel ID_C \parallel AD_C \parallel ID_{TGS} \parallel TS_2 \parallel Lifetime_2])$ $Ticket_V = E(K_V, [K_{C,V} \parallel ID_C \parallel AD_C \parallel ID_V \parallel TS_4 \parallel Lifetime_4])$ $Authenticator_c = E(K_{C,TGS}, [ID_C \parallel AD_C \parallel TS_3])$ (b) Ticket-Granting Service Exchange to obtain service-granting ticket
(5) $C \rightarrow V \ Ticket_V \parallel Authenticator_c$ (6) $V \rightarrow C \ E(K_{C,V}, [TS_5 + 1])$ (for mutual authentication) $Ticket_V = E(K_V, [K_{C,V} \parallel ID_C \parallel AD_C \parallel ID_V \parallel TS_4 \parallel Lifetime_4])$ $Authenticator_c = E(K_{C,V}, [ID_C \parallel AD_C \parallel TS_5])$ (c) Client/Server Authentication Exchange to obtain service

Table 4.1 a shows the technique for distributing the session key. As before, the client sends a message to the AS requesting access to the TGS. The AS responds with a message, encrypted with a key derived from the user's password (K_c) that contains the ticket. The encrypted message also contains a copy of the session key, ($K_{c,tgs}$), where the subscripts indicate that this is a session key for C and TGS. Because this session key is inside the message encrypted with (K_c), only the user's client can read it. The same session key is included in the ticket, which can be read only by the TGS. Thus, the session key has been securely delivered to both C and the TGS.

Note that several additional pieces of information have been added to this first phase of the dialogue. Message (1) includes a timestamp, so that the AS knows that the message is timely. Message (2) includes several elements of the ticket in a form accessible to C. This enables C to confirm that this ticket is for the TGS and to learn its expiration time. Note that the ticket does not prove anyone's identity but is a way to distribute keys securely.

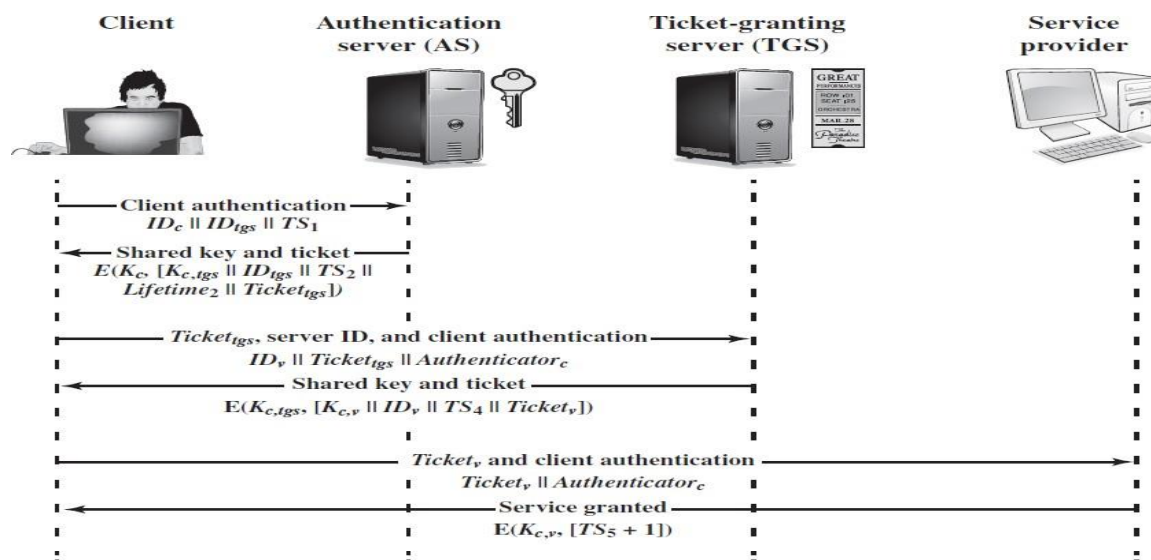
It is the authenticator that proves the client's identity.

C sends the TGS a message that includes the ticket plus the ID of the requested service (message (3) in Table b). In addition, C transmits an authenticator, which includes the ID and address of C's user and a timestamp. Unlike the ticket, which is reusable, the authenticator is intended for use only once and has a very short lifetime. The TGS can decrypt the ticket with the key that it shares with the AS. This ticket indicates that user C has been provided with the session key $K_{c, tgs}$. In effect, the ticket says, "Anyone who uses $K_{c, tgs}$ must be C." The TGS uses the session key to decrypt the authenticator.

The reply from the TGS, in message (4), follows the form of message (2). C now has a reusable service-granting ticket for V. When C presents this ticket, as shown in message (5), it also sends an authenticator. The server can decrypt the ticket, recover the session key, and decrypt the authenticator. If mutual authentication is required, the server can reply as shown in message (6).

Finally, at the conclusion of this process, the client and server share a secret key.

This key can be used to encrypt future messages between the two or to exchange a new random session key for that purpose Fig 4.4.



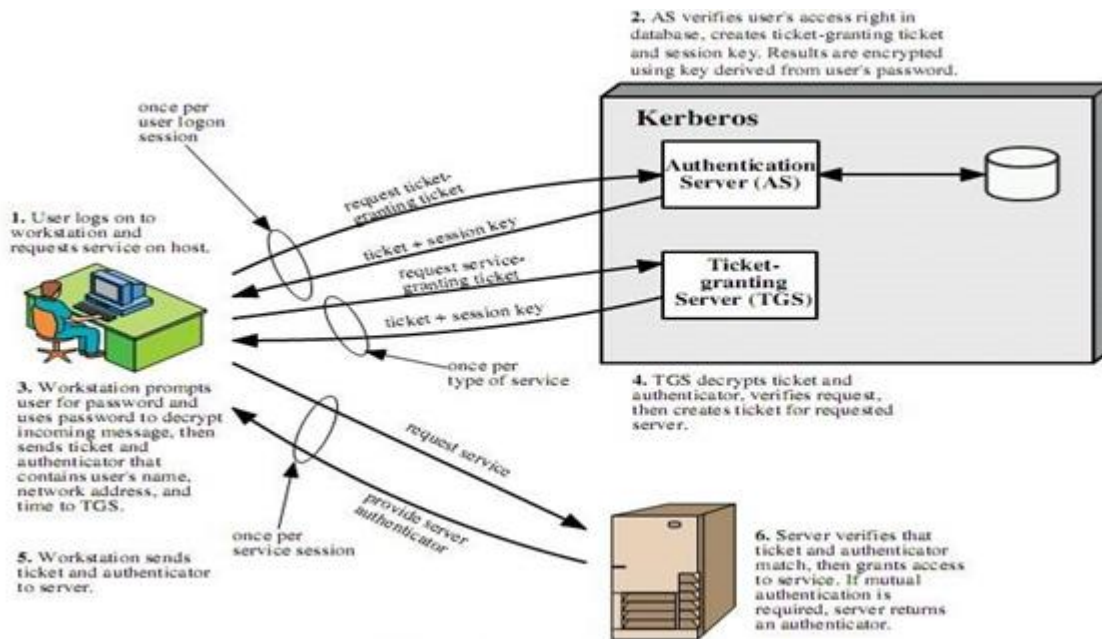


Fig 4.4: Overview of KERBEROS

Message (1)	Client requests ticket-granting ticket.
ID_C	Tells AS identity of user from this client.
ID_{TGS}	Tells AS that user requests access to TGS.
TS_1	Allows AS to verify that client's clock is synchronized with that of AS.
Message (2)	AS returns ticket-granting ticket.
K_c	Encryption is based on user's password, enabling AS and client to verify password, and protecting contents of message (2).
$K_{c, tgs}$	Copy of session key accessible to client created by AS to permit secure exchange between client and TGS without requiring them to share a permanent key.
ID_{TGS}	Confirms that this ticket is for the TGS.
TS_2	Informs client of time this ticket was issued.
$Lifetime_2$	Informs client of the lifetime of this ticket.
$Ticket_{TGS}$	Ticket to be used by client to access TGS.

(a) Authentication Service Exchange

Message (3)	Client requests service-granting ticket.
ID_V	Tells TGS that user requests access to server V.
$Ticket_{tgs}$	Assures TGS that this user has been authenticated by AS.
$Authenticator_c$	Generated by client to validate ticket.
Message (4)	TGS returns service-granting ticket.
$K_{c,tgs}$	Key shared only by C and TGS protects contents of message (4).
$K_{c,v}$	Copy of session key accessible to client created by TGS to permit secure exchange between client and server without requiring them to share a permanent key.
ID_V	Confirms that this ticket is for server V.
TS_4	Informs client of time this ticket was issued.
$Ticket_V$	Ticket to be used by client to access server V.
$Ticket_{tgs}$	Reusable so that user does not have to reenter password.
K_{tgs}	Ticket is encrypted with key known only to AS and TGS, to prevent tampering.
$K_{c,tgs}$	Copy of session key accessible to TGS used to decrypt authenticator, thereby authenticating ticket.
ID_C	Indicates the rightful owner of this ticket.
AD_C	Prevents use of ticket from workstation other than one that initially requested the ticket.
ID_{tgs}	Assures server that it has decrypted ticket properly.
TS_2	Informs TGS of time this ticket was issued.
$Lifetime_2$	Prevents replay after ticket has expired.
$Authenticator_c$	Assures TGS that the ticket presenter is the same as the client for whom the ticket was issued has very short lifetime to prevent replay.
$K_{c,tgs}$	Authenticator is encrypted with key known only to client and TGS, to prevent tampering.
ID_C	Must match ID in ticket to authenticate ticket.
AD_C	Must match address in ticket to authenticate ticket.
TS_3	Informs TGS of time this authenticator was generated.

(b) Ticket-Granting Service Exchange

Message (5)	Client requests service.
$Ticket_V$	Assures server that this user has been authenticated by AS.
$Authenticator_c$	Generated by client to validate ticket.
Message (6)	Optional authentication of server to client.
$K_{c,v}$	Assures C that this message is from V.
$TS_5 + 1$	Assures C that this is not a replay of an old reply.
$Ticket_v$	Reusable so that client does not need to request a new ticket from TGS for each access to the same server.
K_v	Ticket is encrypted with key known only to TGS and server, to prevent tampering.
$K_{c,v}$	Copy of session key accessible to client; used to decrypt authenticator, thereby authenticating ticket.
ID_C	Indicates the rightful owner of this ticket.
AD_C	Prevents use of ticket from workstation other than one that initially requested the ticket.
ID_V	Assures server that it has decrypted ticket properly.
TS_4	Informs server of time this ticket was issued.
$Lifetime_4$	Prevents replay after ticket has expired.
$Authenticator_c$	Assures server that the ticket presenter is the same as the client for whom the ticket was issued; has very short lifetime to prevent replay.
$K_{c,v}$	Authenticator is encrypted with key known only to client and server, to prevent tampering.
ID_C	Must match ID in ticket to authenticate ticket.
AD_C	Must match address in ticket to authenticate ticket.
TS_5	Informs server of time this authenticator was generated.

(c) Client/Server Authentication Exchange

4.4 Kerberos Realms:

A full-service Kerberos environment consisting of a Kerberos server, a number of clients, and a number of application servers requires the following:

1. The Kerberos server must have the user ID and hashed passwords of all participating users in its database. All users are registered with the Kerberos server.
2. The Kerberos server must share a secret key with each server. All servers are registered

3. with the Kerberos server.
4. The Kerberos server in each interoperating realm shares a secret key with the server in the other realm. The two Kerberos servers are registered with each other.

A full-service Kerberos environment consisting of a Kerberos server, a number of clients, and a number of application servers is referred to as a Kerberos realm. A Kerberos realm is a set of managed nodes that share the same Kerberos database, and are part of the same administrative domain. If have multiple realms, their Kerberos servers must share keys and trust each other.

The details of the exchanges illustrated in below Fig 4.5 are as follows

- (1) $C \rightarrow AS:$ $ID_c \parallel ID_{tgs} \parallel TS_1$
- (2) $AS \rightarrow C:$ $E(K_{c,tgs}, [K_{c,tgs} \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel Ticket_{tgs}])$
- (3) $C \rightarrow TGS:$ $ID_{tgsrem} \parallel Ticket_{tgs} \parallel Authenticator_c$
- (4) $TGS \rightarrow C:$ $E(K_{c,tgs}, [K_{c,tgsrem} \parallel ID_{tgsrem} \parallel TS_4 \parallel Ticket_{tgsrem}])$
- (5) $C \rightarrow TGS_{rem}:$ $ID_{vrem} \parallel Ticket_{tgsrem} \parallel Authenticator_c$
- (6) $TGS_{rem} \rightarrow C:$ $E(K_{c,tgsrem}, [K_{c,vrem} \parallel ID_{vrem} \parallel TS_6 \parallel Ticket_{vrem}])$
- (7) $C \rightarrow V_{rem}:$ $Ticket_{vrem} \parallel Authenticator_c$

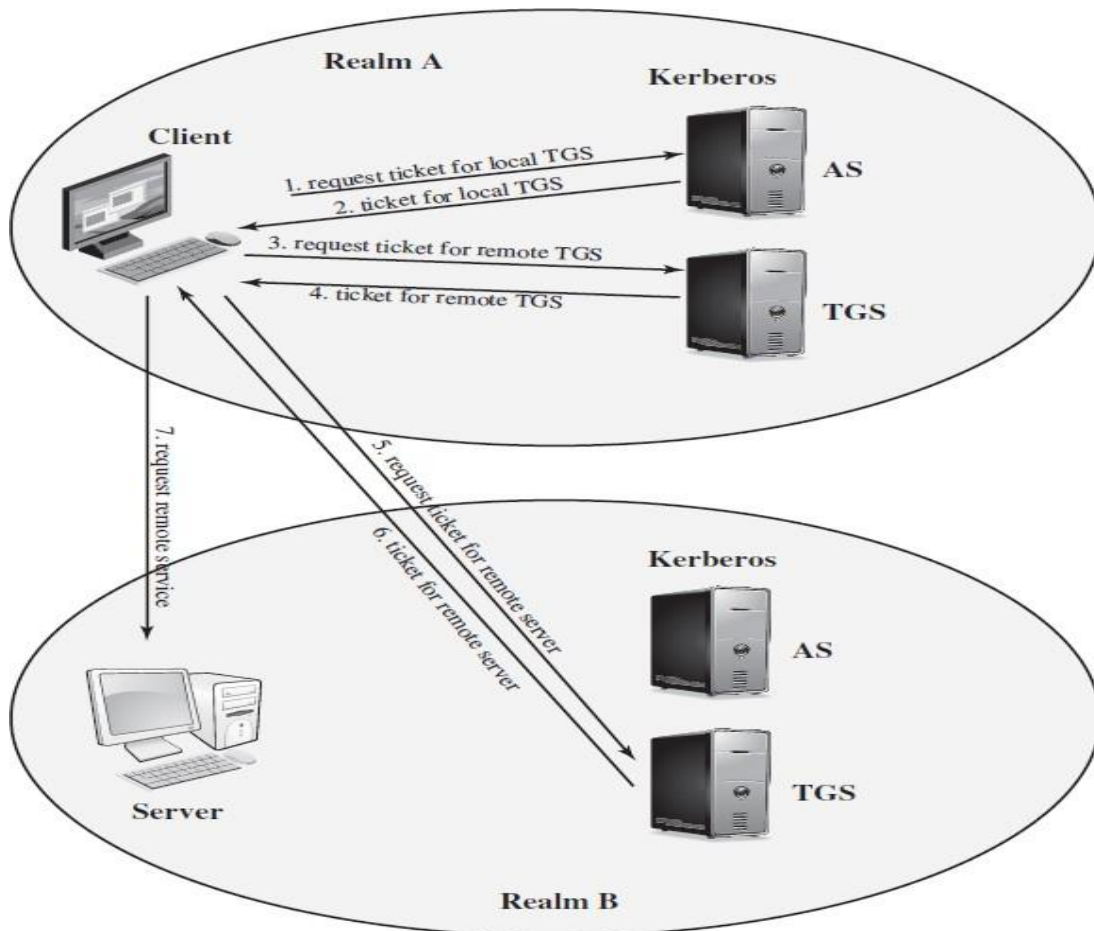


Fig 4.5: Exchanges illustrated

4.5 The Version 5 Authentication Dialogue

Table 4.2: Message Exchanges

- (1) $C \rightarrow AS$ $Options \parallel ID_C \parallel Realm_c \parallel ID_{TGS} \parallel Times \parallel Nonce_1$
 (2) $AS \rightarrow C$ $Realm_c \parallel ID_C \parallel Ticket_{TGS} \parallel E(K_{c,TGS}, [K_{c,TGS} \parallel Times \parallel Nonce_1 \parallel Realm_{TGS} \parallel ID_{TGS}])$
 $Ticket_{TGS} = E(K_{TGS}, [Flags \parallel K_{c,TGS} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$

(a) Authentication Service Exchange to obtain ticket-granting ticket

- (3) $C \rightarrow TGS$ $Options \parallel ID_v \parallel Times \parallel Nonce_2 \parallel Ticket_{TGS} \parallel Authenticator_c$
 (4) $TGS \rightarrow C$ $Realm_c \parallel ID_C \parallel Ticket_v \parallel E(K_{c,TGS}, [K_{c,v} \parallel Times \parallel Nonce_2 \parallel Realm_v \parallel ID_v])$
 $Ticket_{TGS} = E(K_{TGS}, [Flags \parallel K_{c,TGS} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$
 $Ticket_v = E(K_v, [Flags \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$
 $Authenticator_c = E(K_{c,TGS}, [ID_C \parallel Realm_c \parallel TS_1])$

(b) Ticket-Granting Service Exchange to obtain service-granting ticket

- (5) $C \rightarrow V$ $Options \parallel Ticket_v \parallel Authenticator_c$
 (6) $V \rightarrow C$ $E_{K_{c,v}}[TS_2 \parallel Subkey \parallel Seq \#]$
 $Ticket_v = E(K_v, [Flag \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel AD_C \parallel Times])$
 $Authenticator_c = E(K_{c,v}, [ID_C \parallel Relam_c \parallel TS_2 \parallel Subkey \parallel Seq \#])$

(c) Client/Server Authentication Exchange to obtain service

INITIAL	This ticket was issued using the AS protocol and not issued based on a ticket-granting ticket.
PRE-AUTHENT	During initial authentication, the client was authenticated by the KDC before a ticket was issued.
HW-AUTHENT	The protocol employed for initial authentication required the use of hardware expected to be possessed solely by the named client.
RENEWABLE	Tells TGS that this ticket can be used to obtain a replacement ticket that expires at a later date.
MAY-POSTDATE	Tells TGS that a postdated ticket may be issued based on this ticket-granting ticket.
POSTDATED	Indicates that this ticket has been postdated; the end server can check the authtime field to see when the original authentication occurred.
INVALID	This ticket is invalid and must be validated by the KDC before use.
PROXIABLE	Tells TGS that a new service-granting ticket with a different network address may be issued based on the presented ticket.
PROXY	Indicates that this ticket is a proxy.
FORWARDABLE	Tells TGS that a new ticket-granting ticket with a different network address may be issued based on this ticket-granting ticket.
FORWARDED	Indicates that this ticket has either been forwarded or was issued based on authentication involving a forwarded ticket-granting ticket.

- Message (1) is a client request for a ticket-granting ticket Table 4.2. As before, it includes the ID of the user and the TGS. The following new elements are added:
 - Realm: Indicates realm of user
 - Options: Used to request that certain flags be set in the returned ticket
 - Times: Used by the client to request the following time settings in the ticket:
 - from: the desired start time for the requested ticket
 - till: the requested expiration time for the requested ticket
 - rtime: requested renew-till time
 - Nonce: A random value to be repeated in message (2) to assure that the response is fresh and has not been replayed by an opponent.
- Message (2) returns a ticket-granting ticket, identifying information for the client, and a block encrypted using the encryption key based on the user's password. This block includes the session key to be used between the client and the TGS, times specified in message (1), the nonce from message (1), and TGS identifying information. The ticket itself includes the session key, identifying information for the client, the requested time values, and flags that reflect the status of this ticket and the requested options. These flags introduce significant new functionality to version 5.
- Message (3) for both versions includes an authenticator, a ticket, and the name of the requested service. In addition, version 5 includes requested times and options for the ticket and a nonce—all with functions similar to those of message 1).
- Message (4) has the same structure as message (2). It returns a ticket plus information needed by the client, with the information encrypted using the session key now shared by the client and the TGS.
- Finally, for the client/server authentication exchange, several new features appear in version 5. In message (5), the client may request as an option that mutual authentication is required. The authenticator includes several new fields:
 - Subkey: The client's choice for an encryption key to be used to protect this specific application session. If this field is omitted, the session key from the ticket (K_c, v) is used.
 - Sequence number: An optional field that specifies the starting sequence number to be used

The server for messages sent to the client during this session. Messages may be sequence numbered to detect replays.

4.6 DIFFERENCES BETWEEN VERSIONS 4 AND 5

Version 5 is intended to address the limitations of version 4 in two areas: environmental shortcomings and technical deficiencies.

Environmental shortcomings

1. Encryption system dependence: Version 4 requires the use of DES. Export restriction on DES as well as doubts about the strength of DES were thus of concern. In version 5, ciphertext is tagged with an encryption-type identifier so that any encryption technique may be used.

2. Internet protocol dependence: Version 4 requires the use of Internet Protocol (IP) addresses. Other address types, such as the ISO network address, are not accommodated. Version 5 network addresses are tagged with type and length, allowing any network address type to be used.

3. Message byte ordering: In version 4, the sender of a message employs a byte ordering of its own choosing and tags the message to indicate least significant byte in lowest address or most significant byte in lowest address. This technique works but does not follow established conventions. In version 5, all message structures are defined using Abstract Syntax Notation One (ASN.1) and Basic Encoding Rules (BER), which provide an unambiguous byte ordering.

4. Ticket lifetime: Lifetime values in version 4 are encoded in an 8-bit quantity in units of five minutes. Thus, the maximum lifetime that can be expressed is $2^8 * 5 = 1280$ minutes (a little over 21 hours). This may be inadequate for some applications. In version 5, tickets include an explicit start time and end time, allowing tickets with arbitrary lifetimes.

5. Authentication forwarding: Version 4 does not allow credentials issued to one client to be forwarded to some other host and used by some other client. This capability would enable a client to access a server and have that server access another server on behalf of the client. For example, a client issues a request to a print server that then accesses the client's file from a file server, using the client's credentials for access. Version 5 provides this capability.

6. Interrealm authentication: In version 4, interoperability among N realms requires on the order of N^2 Kerberos-to-Kerberos relationships, as described earlier. Version 5 supports a method that requires fewer relationships, as described shortly.

4.6.1 Technical deficiencies:

1. Double encryption: [messages (2) and (4)] that tickets provided to clients are encrypted twice - once with the secret key of the target server and then again with a secret key known to the client. The second encryption is not necessary and is computationally wasteful.

2. PCBC encryption: Encryption in version 4 makes use of a nonstandard mode of DES known as propagating cipher block chaining (PCBC). It has been demonstrated that this mode is vulnerable to an attack involving the interchange of ciphertext blocks. Version 5 provides explicit integrity mechanisms, a checksum or hash code is attached to the message prior to encryption using CBC.

3. Session keys: Each ticket includes a session key that is used by the client to encrypt the authenticator sent to the service associated with that ticket. In addition, the session key may subsequently be used by the client and the server to protect messages passed during that session. However, because the same ticket may be used repeatedly to gain service from a particular server, there is the risk that an opponent will replay messages from an old session to the client or the server. In version 5, it is possible for a client and server to negotiate a subsession key, which is to be used only for that one connection.

4. Password attacks: Both versions are vulnerable to a password attack. The message from the AS to the client includes material encrypted with a key based on the client's password. An opponent can capture this message and attempt to decrypt it by trying various passwords. If the result of a test decryption is of the proper form, then the opponent has discovered the client's password and may subsequently use it to gain authentication credentials from Kerberos. Version 5 does provide a mechanism known as pre authentication, which should make password attacks more difficult, but it does not prevent them.

4.7 Electronic Mail Security

- In virtually all distributed environments, electronic mail is the most heavily used network-based application.
- Users expect to be able to, and do, send e-mail to others who are connected directly or indirectly to the Internet, regardless of host operating system or communications suite.

4.7.1 Pretty Good Privacy

- ✓ PGP is an open-source, freely available software package for e-mail security.
- ✓ It provides **authentication** through the use of **digital signature**, **confidentiality** through the use of **symmetric block encryption**, **compression** using the **ZIP algorithm**, and **e-mail compatibility** using the **radix-64 encoding scheme**.
- ✓ PGP was developed by **Phil Zimmermann**.
- ✓ Zimmermann has done the following:
 1. Selected the best available **cryptographic algorithms** as building blocks.
 2. Integrated these algorithms into a general-purpose application that is independent of operating system and processor and that is based on a small set of easy-to-use commands.
 3. Made the package and its documentation, including the source code, freely available via the Internet, bulletin boards, and commercial networks such as AOL (America On Line).
 4. Entered into an agreement with a company (Via crypt, now Network Associates) to provide a fully compatible, low-cost commercial version of PGP.

PGP has grown explosively and is now widely used because of following reasons:

1. It is available free worldwide in versions that run on a variety of platforms, including Windows, UNIX, Macintosh, and many more.
2. It is based on algorithms that are considered extremely secure such as **RSA, DSS, and Diffie-Hellman for public-key encryption; CAST-128, IDEA, and 3DES for symmetric encryption; and SHA-1 for hash coding**.
3. It has a wide range of applicability, from corporations that wish to select and enforce a standardized scheme for encrypting files and messages to individuals who wish to communicate securely with others worldwide over the Internet and other networks.
4. It was not developed by, nor is it controlled by, any governmental or standards organization.
5. PGP is now on an Internet standards track (RFC 3156; *MIME Security with OpenPGP*).

Notations

K_S	:	Session key used in Symmetric Encryption Scheme
PR_a	:	Private key of User A, used in Public Encryption Scheme
PU_a	:	Public key of User A, used in Public Encryption
Scheme EP	:	Public Encryption
DP	:	Public Decryption
EC	:	Symmetric
Encryption DC	:	Symmetric
Decryption H	:	Hash
Function		
$\parallel$	:	Concatenation
Z	:	Compression
R64	:	Radix 64 conversion

Operational Description

The actual operation of PGP consists of **five series**:

- **Authentication**
- **Confidentiality**
- **Compression**
- **E-mail compatibility**
- **Segmentation**

Authentication

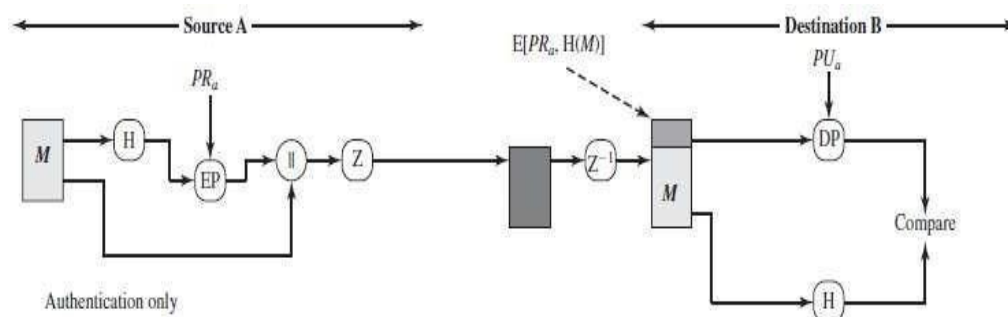


Fig 4.6: Authentication

- ✓ It is provided through the digital signatures Fig 4.6 . The sequence is as follows
 1. The sender creates a message.
 2. **SHA-1** is used to generate a **160-bit hash code of the message**.
 3. The **hash code is encrypted with RSA** using the **sender's private key**, and the result is prepended to the message.
 4. The **receiver uses RSA with the sender's public key** to decrypt and recover

the hash code.

5. The receiver generates a **new hash code for the message and compares it with the decrypted hash code**. If the two match, the message is accepted as authentic.

- ✓ The combination of SHA-1 and RSA provides an effective digital signature scheme.
- ✓ Because of the strength of RSA, the recipient is assured that only the possessor of the matching private key can generate the signature.
- ✓ Because of the strength of SHA-1, the recipient is assured that no one else could generate a new message that matches the hash code and, hence, the signature of the original message
- ✓ Detached signatures are supported. A detached signature may be stored and transmitted separately from the message it signs.
- ✓ Finally, detached signatures can be used when more than one party must sign a document, such as a legal contract.

2. Confidentiality

It is provided by encrypting messages to be transmitted or to be stored locally as files. The sequence can be described as follows Fig 4.7.

1. The sender generates a message and a random **128-bit number** to be used as a **session key** for this message only.
2. The message is encrypted using **CAST-128 (or IDEA or 3DES) with the session key**.
3. The **session key is encrypted with RSA** using the **recipient's public key** and is prepended to the message.
4. The receiver uses RSA with its private key to decrypt and recover the session key.
5. The session key is used to decrypt the message.

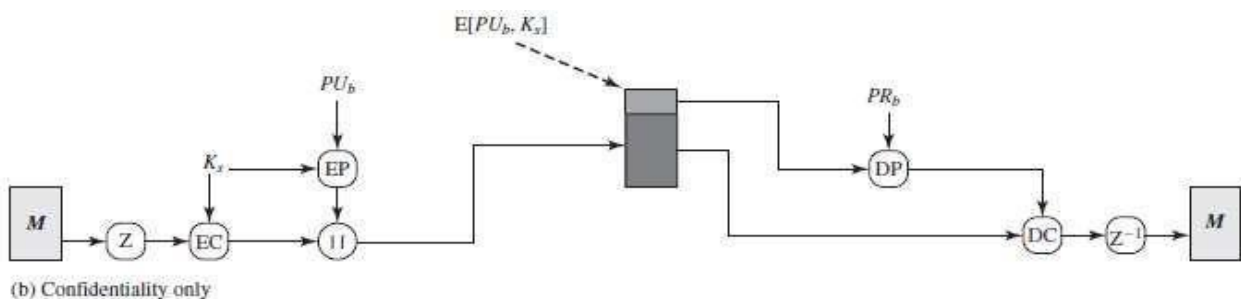


Fig 4.7: Confidentiality

- ✓ Instead of using the RSA also Diffie Hellman can be used.
- ✓ Diffie- Hellman is a key exchange algorithm.
- ✓ In fact, PGP uses a variant of Diffie-Hellman that does provide encryption/decryption, known as ElGamal.

3. Confidentiality and Authentication

1. Here both services (Confidentiality and Authentication) may be used for the same message Fig 4.8.
2. First, a signature is generated for the plaintext message and prepended to the message.
3. Then the plaintext message plus signature is encrypted using CAST-128 (or IDEA or 3DES), and the session key is encrypted using RSA (or ElGamal).

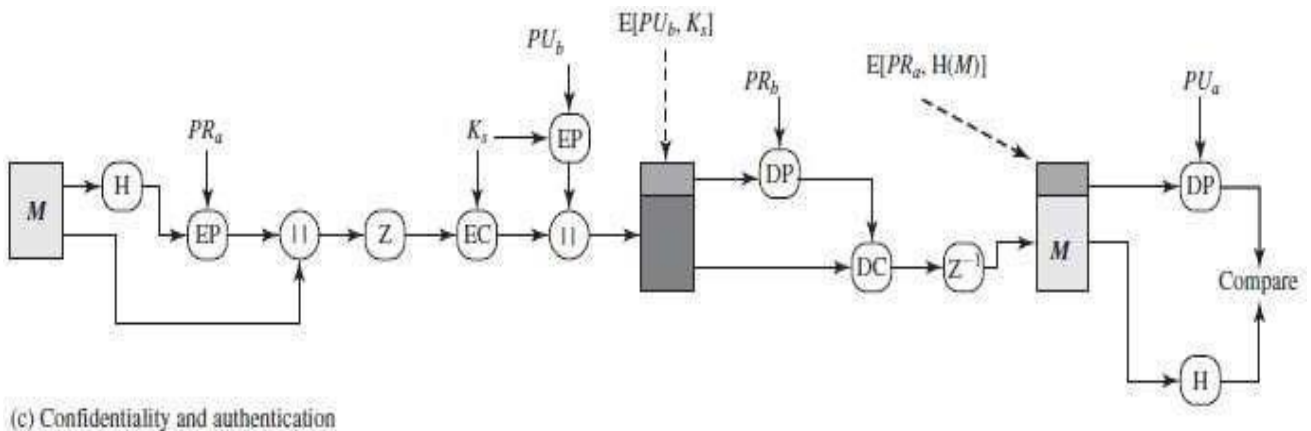


Fig 4.8 Confidentiality and Authentication

4. Compression

- ✓ PGP compresses the message after applying the signature but before encryption.
- ✓ The signature is generated before compression for two reasons:
- ✓ It is preferable to sign an uncompressed message so that one can store only the uncompressed message together with the signature for future verification.
- ✓ If one signed a compressed document, then it would be necessary either to store a compressed version of the message for later verification or to recompress the message when verification is required.
- ✓ However, these different compression algorithms are interoperable because any version of the algorithm can correctly decompress the output of any other version.
- ✓ Applying the hash function and signature after compression would constrain all PGP implementations to the same version of the compression algorithm.
- ✓ Message encryption is applied after compression to strengthen cryptographic security. Because the compressed message has less redundancy than the original plaintext, cryptanalysis is more difficult. The compression algorithm used is ZIP

5. Email Compatibility

1. When PGP is used, at least part of the block to be transmitted is encrypted.
2. If only the **signature service is used, then the message digest(hash code) is encrypted** (with the sender's private key).
3. If the confidentiality service is used, the **message plus signature (if present) are encrypted (with a one-time symmetric key)**. Thus, part or the entire resulting block consists of a stream of arbitrary **8-bit octets**.
4. PGP provides the service of **converting the raw 8-bit binary stream to a stream of printable ASCII characters**.
5. The scheme used for this purpose is **radix-64 conversion**.
6. Each group of **three octets of binary data is mapped into four ASCII characters**.
7. The **use of radix 64 expands a message by 33%**.
8. Each group of three octets of binary data is mapped into four ASCII characters.

6. Segmentation and Reassembly

1. Email facilities are often restricted to a **maximum message length**.
2. To accommodate this restriction, the PGP subdivides the message that is too large into segments that are small enough to send via email.
3. Thus the session key component, and signature component appear only once at the beginning of the first segment.
4. At the receiving end, PGP strips off all email headers and reassemble the entire original block of data.

The below Fig 4.9 shows the relationship among the four services

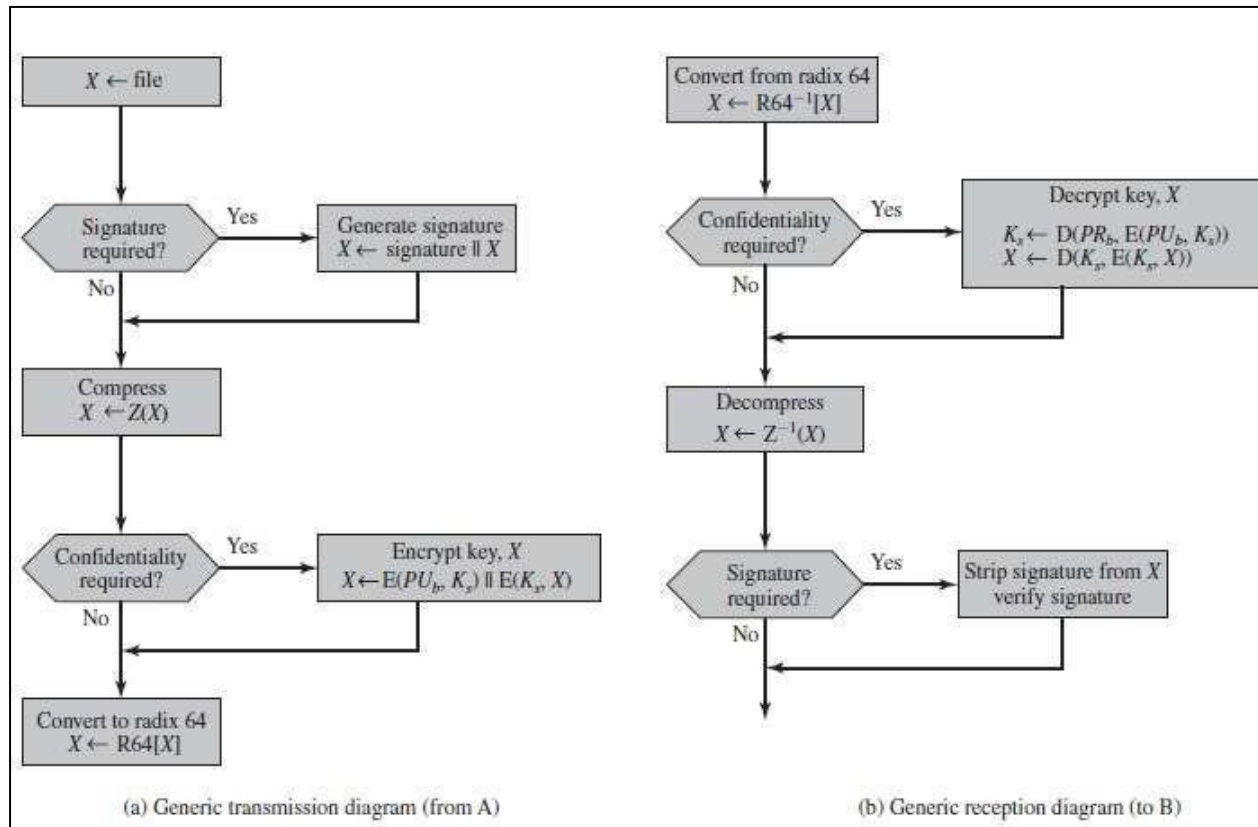


Fig 4.9: Transmission and Reception of PGP Message

Transmission

- ✓ **On transmission** (if it is required), a signature is generated using a hash code of the uncompressed plaintext.
- ✓ Then the plaintext (plus signature if present) is compressed.
- ✓ Next, if confidentiality is required, the block (compressed plaintext or compressed signature plus plaintext) is encrypted and prepended with the public-key encrypted symmetric encryption key.
- ✓ Finally, the entire block is converted to radix-64 format.

Reception

- ✓ On reception, the incoming block is first converted back from radix-64 format to binary.
- ✓ Then, if the message is encrypted, the recipient recovers the session key and decrypts the message.
- ✓ The resulting block is then decompressed.
- ✓ If the message is signed, the recipient recovers the transmitted hash code and compares it to its own calculation of the hash code.

4.8 S/MIME (Security/Multipurpose Internet Mail Extension)

S/MIME is very similar to PGP. Both offer the ability to sign and/or encrypt messages.

- ✓ S/MIME (Secure/Multipurpose Internet Mail Extensions) is a security enhancement to the MIME internet e-mail format standard, based on RSA data security.
- ✓ S/MIME provides the following cryptographic security services for electronic messaging applications:
 - Authentication
 - Message integrity
 - Non-repudiation of origin using digital signatures
 - Data confidentiality using encryption

RFC 822

- ✓ RFC 822 defines a format for text messages that are sent using electronic mail. It has been the standard for Internet-based text mail messages and remains in common use.
- ✓ In the RFC 822 context, messages are viewed as having an envelope and contents.
- ✓ The envelope contains whatever information is needed to accomplish transmission and delivery.
- ✓ The overall structure of a message that conforms to RFC 822 is very simple.
- ✓ A message consists of some number of header lines (the header) followed by unrestricted text (the body). The header is separated from the body by a blank line
- ✓ A header line consists of a keyword, followed by a colon, followed by the keyword's arguments; the format allows a long line to be broken up into several lines.
- ✓ The most frequently used keywords are From, To, Subject, and Date. Here is an example message.

Example:

Date: October 8, 2009 2:15:49 PM EDT
From: William Stallings <ws@shore.net>
Subject: The Syntax in RFC822
To: Smith@Other-host.com
Cc: Jones@Yet-Another-Host.com

Hello. This section begins the **actual message body, which is delimited from the message heading by a blank line.**

4.8.1 Multipurpose Internet Mail Extensions (MIME)

- ✓ Multipurpose Internet Mail Extension (MIME) is an extension to the RFC 822 framework that is intended to address some of the problems and limitations of the use of Simple Mail Transfer Protocol (SMTP).

- ✓ They are
 1. SMTP cannot transmit executable files or other binary objects. A number of schemes are in use for converting binary files into a text form that can be used by SMTP mail systems, including the popular UNIX UUencode/Uudecode scheme. However, none of these is a standard or even a de facto standard.
 2. SMTP cannot transmit text data that includes national language characters, because these are represented by 8-bit codes with values of 128 decimal or higher, and SMTP is limited to 7-bit ASCII.
 3. SMTP servers may reject mail message over a certain size.
 4. SMTP gateways that translate between ASCII and the character code EBCDIC do not use a consistent set of mappings, resulting in translation problems.
 5. SMTP gateways to X.400 electronic mail networks cannot handle non textual data included in X.400 messages.
 6. Some SMTP implementations do not adhere completely to the SMTP standards defined in RFC 821
- ✓ Common problems include:
 - Deletion, addition, or reordering of carriage return and linefeed
 - Truncating or wrapping lines longer than 76 characters
 - Removal of trailing white space (tab and space characters)
 - Padding of lines in a message to the same length
 - Conversion of tab characters into multiple space characters.

OVERVIEW

The MIME specification includes the following elements

1. Five new message header fields are defined, which may be included in an RFC 822 header. These fields provide information about the body of the message.
2. A number of content formats are defined, thus standardizing representations that support multimedia electronic mail.
3. Transfer encodings are defined that enable the conversion of any content format into a form that is protected from alteration by the mail system.

(Explain the header fields of MIME protocol...)

The five header fields defined in MIME are

- **MIME-Version:** Must have the parameter value 1.0. This field indicates that the message conforms to RFCs 2045 and 2046.
- **Content-Type:** Describes the data contained in the body with sufficient detail that the receiving user agent can pick an appropriate agent or mechanism to represent the data to the user or otherwise deal with the data in an appropriate manner.

- **Content-Transfer-Encoding:** Indicates the type of transformation that has been used to represent the body of the message in a way that is acceptable for mail transport.
- **Content-ID:** Used to identify MIME entities uniquely in multiple contexts.
- **Content-Description:** A text description of the object with the body; this is useful when the object is not readable (e.g., audio data).

(Explain **MIME** Content types ?)

MIME CONTENT TYPES:

- ✓ The bulk of the MIME specification is concerned with the definition of a variety of content types.
- ✓ There are seven different major types of content and a total of 15 subtypes. In general, a content type declares the general type of data, and the subtype specifies a particular format for that type of data.
- ✓ For the text type of body, no special software is required to get the full meaning of the text aside from support of the indicated character set.
- ✓ The primary subtype is plain text, which is simply a string of ASCII characters or ISO 8859 characters. The enriched subtype allows greater formatting flexibility.
- ✓ The multipart type indicates that the body contains multiple, independent parts.
- ✓ The Content-Type header field includes a parameter (called a boundary) that defines the delimiter between body parts.
- ✓ Each boundary starts on a new line and consists of two hyphens followed by the boundary value. The final boundary, which indicates the end of the last part, also has a suffix of two hyphens.
- ✓ Within each part, there may be an optional ordinary MIME header.

Example:

From: Nathaniel Borenstein <nsb@bellcore.com>

To: Ned Freed <ned@innosoft.com>

Subject: Sample message

MIME-Version: 1.0

Content-type: multipart/mixed; boundary="simple boundary"

This is the preamble. It is to be ignored, though it is a handy place for mail composers to include an explanatory note to non-MIME conformant readers.

—**simple boundary**

This is implicitly typed plain ASCII text. It does NOT end with a line break.

—**simple boundary**

Content-type: text/plain; charset=us-ascii

This is explicitly typed plain ASCII text. It DOES end with a linebreak.

Table 4.3: MIME Content Types

Type	Subtype	Description
Text	Plain	Unformatted text; may be ASCII or ISO 8859.
	Enriched	Provides greater format flexibility.
Multipart	Mixed	The different parts are independent but are to be transmitted together. They should be presented to the receiver in the order that they appear in the mail message.
	Parallel	Differs from Mixed only in that no order is defined for delivering the parts to the receiver.
	Alternative	The different parts are alternative versions of the same information. They are ordered in increasing faithfulness to the original, and the recipient's mail system should display the "best" version to the user.
	Digest	Similar to Mixed, but the default type/subtype of each part is message/rfc822.
Message	rfc822	The body is itself an encapsulated message that conforms to RFC 822.
	Partial	Used to allow fragmentation of large mail items, in a way that is transparent to the recipient.
	External-body	Contains a pointer to an object that exists elsewhere.
Image	jpeg	The image is in JPEG format, JFIF encoding.
	gif	The image is in GIF format.
Video	mpeg	MPEG format.
Audio	Basic	Single-channel 8-bit ISDN mu-law encoding at a sample rate of 8 kHz.
Application	PostScript	Adobe Postscript format.
	octet-stream	General binary data consisting of 8-bit bytes.

- ✓ There are **four subtypes of the multipart type**, all of which have the same overall syntax.
- 1. The **multipart/mixed subtype** is used when there are multiple independent body parts that need to be bundled in a particular order.
- 2. **The multipart/parallel subtype**, the order of the parts is not significant. If the recipient's system is appropriate, the multiple parts can be presented in parallel.
- 3. For the **multipart/alternative subtype**, the various parts are different representations of the same information.

4. The **multipart/digest subtype** is used when each of the body parts is interpreted as an RFC 822 message with headers. This subtype enables the construction of a message whose parts are individual messages.

Example of multipart/alternative subtype:

```
From: Nathaniel Borenstein <nsb@bellcore.com>
To: Ned Freed <ned@innosoft.com>
Subject: Formatted text mail
MIME-Version: 1.0
Content-Type: multipart/alternative;
boundary=boundary42
—boundary42
Content-Type: text/plain; charset=us-ascii
...plain text version of message goes here....
—boundary42
Content-Type: text/enriched
.... RFC 1896 text/enriched version of same message
goes here ...
—boundary42—
```

There are three sub types in message **type** in MIME.

1. The **message/rfc822 subtype**: indicates that the body is an entire message, including header and body.
 2. **The message/partial subtype** enables fragmentation of a large message into a number of parts, which must be reassembled at the destination. For this subtype, three parameters are specified in the Content-Type: Message/Partial field: an id common to all fragments of the same message, a sequence number unique to each fragment, and the total number of fragments.
 3. **The message/external-body subtype**: indicates that the actual data to be conveyed in this message are not contained in the body. Instead, the body contains the information needed to access the data
- ✓ **The application type** refers to other kinds of data, typically either uninterpreted binary data or information to be processed by a mail-based application. The quoted-printable transfer encoding is useful when the data consists largely of octets that correspond to printable ASCII characters.

(Explain MIME Transfer Encodings ?)

4.8.2 MIME TRANSFER ENCODINGS

- ✓ The other major component of the MIME specification, in addition to content type specification, is a definition of transfer encodings for message bodies.

- ✓ The Content-Transfer-Encoding field can actually take on **six values**, as listed in below Table. However, three of these values (7bit, 8bit, and binary) indicate that no encoding has been done but provide some information about the nature of the data.
- ✓ For SMTP transfer, it is safe to use the 7bit form.
- ✓ The 8bit and binary forms may be usable in other mail transport contexts.
- ✓ x-token Encoding, which indicates that some other encoding scheme is used for which a name is to be supplied. This could be a vendor-specific or application-specific scheme.
- ✓ The **two actual encoding schemes defined are quoted-printable and base64**.
- ✓ Two schemes are defined to provide a choice between a transfer technique that is essentially human readable and one that is safe for all types of data in a way that is reasonably compact.
- ✓ The **quoted-printable** transfer encoding is useful when the data consists largely of **octets** that correspond to **printable ASCII characters Table 4.4**.
- ✓ In essence, it represents unsafe characters by the hexadecimal representation of their code and introduces reversible (soft) line breaks to limit message lines to 76 characters.
- ✓ The **base64 transfer encoding**, also known as **radix-64 encoding**, is a common one for **encoding arbitrary binary data** in such a way as to be invulnerable to the processing by mail-transport programs.

Table 4.4: MIME Transfer encoding

7bit	The data are all represented by short lines of ASCII characters.
8bit	The lines are short, but there may be non-ASCII characters (octets with the high-order bit set).
binary	Not only may non-ASCII characters be present, but the lines are not necessarily short enough for SMTP transport.
quoted-printable	Encodes the data in such a way that if the data being encoded are mostly ASCII text, the encoded form of the data remains largely recognizable by humans.
base64	Encodes data by mapping 6-bit blocks of input to 8-bit blocks of output, all of which are printable ASCII characters.
x-token	A named nonstandard encoding.

4.9 S/MIME Functionality:

(Describe S/MIME functionality.)

In terms of general functionality, S/MIME is very similar to PGP. Both offer the ability to sign and/or encrypt messages. In this subsection, we briefly summarize S/MIME capability.

FUNCTIONS S/MIME provides the following functions.

- **Enveloped data:** This consists of encrypted content of any type and encrypted content encryption keys for one or more recipients.
- **Signed data:** A digital signature is formed by taking the message digest of the content

to be signed and then encrypting that with the private key of the signer. The content plus signature are then encoded using base64 encoding. A signed data message can only be viewed by a recipient with S/MIME capability.

- **Clear-signed data:** As with signed data, a digital signature of the content is formed. However, in this case, only the digital signature is encoded using base64. As a result, recipients without S/MIME capability can view the message content, although they cannot verify the signature.
- **Signed and enveloped data:** Signed-only and encrypted-only entities may be nested, so that encrypted data may be signed and signed data or clear-signed data may be encrypted.

4.10 CRYPTOGRAPHIC ALGORITHM

- S/MIME uses the following terminology to specify requirement level:

MUST: The definition is an absolute requirement of the specification. An implementation must include this feature or function to be in conformance with the specification.

SHOULD: There may exist valid reasons in particular circumstances to ignore this feature or function, but it is recommended that an implementation include the feature or function.

- S/MIME incorporates three public-key algorithms.
 - The Digital Signature Standard (DSS) is the preferred algorithm for digital signature.
 - S/MIME lists Diffie-Hellman as the preferred algorithm for encrypting session keys
 - RSA, can be used for both signatures and session key encryption.

Table 4.5: Cryptographic Algorithms – Used in MIME

Function	Requirement
Create a message digest to be used in forming a digital signature.	MUST support SHA-1. Receiver SHOULD support MD5 for backward compatibility.
Encrypt message digest to form a digital signature.	Sending and receiving agents MUST support DSS. Sending agents SHOULD support RSA encryption. Receiving agents SHOULD support verification of RSA signatures with key sizes 512 bits to 1024 bits.
Encrypt session key for transmission with a message.	Sending and receiving agents SHOULD support Diffie-Hellman. Sending and receiving agents MUST support RSA encryption with key sizes 512 bits to 1024 bits.
Encrypt message for transmission with a one-time session key.	Sending and receiving agents MUST support encryption with tripleDES. Sending agents SHOULD support encryption with AES. Sending agents SHOULD support encryption with RC2/40.
Create a message authentication code.	Receiving agents MUST support HMAC with SHA-1. Sending agents SHOULD support HMAC with SHA-1.

- For the hash function used to create the digital signature, the specification requires the 160-bit SHA-1 but recommends receiver support for the 128-bit MD5 for backward compatibility with older versions of S/MIME.
- For message encryption, three-key triple DES (tripleDES) is recommended, but compliant implementations must support 40-bit RC2.
- The following rules, in the following order, should be followed by a sending agent.
 1. If the sending agent has a list of preferred decrypting capabilities from an intended recipient, it SHOULD choose the first (highest preference) capability on the list that it is capable of using.
 2. If the sending agent has no such list of capabilities from an intended recipient but has received one or more messages from the recipient, then the outgoing message SHOULD use the same encryption algorithm as was used on the last signed and encrypted message received from that intended recipient.
 3. If the sending agent has no knowledge about the decryption capabilities of the intended recipient and is willing to risk that the recipient may not be able to decrypt the message, then the sending agent SHOULD use triple DES.
 4. If the sending agent has no knowledge about the decryption capabilities of the intended recipient and is not willing to risk that the recipient may not be able to decrypt the message, then the sending agent MUST use RC2/40.

S/MIME Messages

(Explain S/MIME content types)

- ✓ S/MIME Table 4.6 makes use of a number of new MIME content types shown in table.
- ✓ All of the new application types use the designation PKCS. This refers to a set of public-key cryptography specifications issued by RSA Laboratories.

Table 4.6: S/MIME Content Types

Type	Subtype	smime Parameter	Description
Multipart	Signed		A clear-signed message in two parts: one is the message and the other is the signature.
Application	pkcs7-mime	signedData	A signed S/MIME entity.
	pkcs7-mime	envelopedData	An encrypted S/MIME entity.
	pkcs7-mime	degenerate signedData	An entity containing only public-key certificates.
	pkcs7-mime	CompressedData	A compressed S/MIME entity.
	pkcs7-signature	signedData	The content type of the signature subpart of a multipart/signed message.

Securing a MIME entity:

- ✓ S/MIME secures a MIME entity with a signature encryption, or both. A MIME entity may be an entire message, or if the MIME content type is multipart, then a MIME entity is one or more of the subparts of the message.
- ✓ Then the MIME entity plus some security-related data, such as algorithm identifiers and certificates, are processed by S/MIME to produce what is known as a PKCS object.
- ✓ A PKCS object is then treated as message content and wrapped in MIME. In all cases, the message to be sent is converted to canonical form.

Enveloped Data

- ✓ An application/pkcs7-mime subtype is used for one of four categories of S/MIME processing, each with a unique s/mime-type parameter.
- ✓ The steps for preparing an enveloped Data MIME entity are:
 1. Generate a pseudorandom session key for a particular symmetric encryption algorithm (RC2/40 or triple DES).
 2. For each recipient, encrypt the session key with the recipient's public RSA key.
 3. For each recipient, prepare a block known as **RecipientInfo** that contains an identifier of the recipient's public-key certificate, an identifier of the algorithm used to encrypt the session key, and the encrypted session key.
 4. Encrypt the message content with the session key.
 - ✓ The **RecipientInfo** blocks followed by the encrypted content constitute the enveloped Data.

Signed data

- ✓ The signedData s/mime-type can be used with one or more signers. For clarity, we confine our description to the case of a single digital signature.

The steps for preparing a signedData MIME entity are:

1. Select a message digest algorithm (SHA or MD5).
 2. Compute the message digest (hash function) of the content to be signed.
 3. Encrypt the message digest with the signer's private key.
 4. Prepare a block known as SignerInfo that contains the signer's public key certificate, an identifier of the message digest algorithm, an identifier of the algorithm used to encrypt the message digest, and the encrypted message digest.
- ✓ The signedData entity consists of a series of blocks, including a message digest algorithm identifier, the message being signed, and SignerInfo.

Clear Signing

- ✓ Clear signing is achieved using the multipart content type with a signed subtype.
- ✓ A multipart/signed message has two parts. The first part can be any MIME type but must be prepared so that it will not be altered during transfer from source to destination.
- ✓ This second part has a MIME content type of application and a subtype of pkcs7-signature.

Registration Request

An application or user will apply to a certification authority for a public-key certificate. The application/pkcs10 S/MIME entity is used to transfer a certification request.

4.9 CERTIFICATES-ONLY MESSAGE

A message containing only certificates or a certificate revocation list (CRL) can be sent in response to a registration request. The message is an application/pkcs7-mime type/subtype with an s/mime-type parameter of degenerate.

4.9.1 S/MIME Certificate Processing

S/MIME uses public-key certificates that conform to version 3 of X.509.

USER AGENT ROLE:

An S/MIME user has several key-management functions to perform.

- **Key generation:** The user or some related administrative utility **MUST** be capable of generating separate Diffie-Hellman and DSS key pairs and **SHOULD** be capable of generating RSA key pairs. Each key pair **MUST** be generated from a good source of a non deterministic random input. A user agent **SHOULD** generate RSA key pairs with a length in the range of 768 – 1024 bits and **MUST NOT** generate less than 512 bits.

- **Registration:** A user's public key must be registered with a certification authority in order to receive an X.509 public-key certificate.
- **Certificate storage and retrieval:** A user requires access to a local list of certificates in order to verify incoming signatures and to encrypt outgoing messages.

VeriSign Certificates

- ✓ There are several companies that provide certification authority (CA) services. VeriSign provides a CA service that is intended to be compatible with S/MIME and a variety of other applications.
- ✓ VeriSign issues X.509 certificates with the product name VeriSign Digital ID.
- ✓ The information contained in a Digital ID depends on the type of Digital ID and its use.
- ✓ At a minimum, each Digital ID contains:
 - Owner's public key
 - Owner's name or alias
 - Expiration date of the Digital ID
 - Serial number of the Digital ID
 - Name of the certification authority that issued the Digital ID
 - Digital signature of the certification authority that issued the Digital ID.
- ✓ Digital IDs can also contain user supplied information :
 - Address
 - Email address
 - Basic registration information (Country ZIP code, age and gender)
- ✓ VeriSign provides three levels, or classes, of security for public-key certificates.
 - For Class 1 Digital IDs, VeriSign confirms the user's e-mail address by sending a PIN and Digital ID pick-up information to the e-mail address provided in the application.
 - For Class 2 Digital IDs, VeriSign verifies the information in the application through an automated comparison with a consumer database in addition to performing all of the checking associated with a Class 1 Digital ID.
 - For Class 3 Digital IDs, VeriSign requires a higher level of identity assurance.

Enhanced Security Services:

The three services are:

- **Signed receipts:** A signed receipt may be requested in a SignedData object. e. In essence, the recipient signs the entire original message plus the original (sender's) signature and appends the new signature to form a new S/MIME messages.
- **Security labels:** A security label may be included in the authenticated attributes of a SignedData object. The labels may be used for access control, by indicating which users are permitted access to an object.

- **Secure mailing lists:** When a user sends a message to multiple recipients, a certain amount of per-recipient processing is required, including the use of each recipient's public key. The user can be relieved of this work by employing the services of an S/MIME Mail List Agent (MLA). An MLA can take a single incoming message, perform the recipient-specific encryption for each recipient, and forward the message. The originator of a message need only send the message to the MLA with encryption performed using the MLA's public key.

4.10 Domain Keys Identified Mail (DKIM)

- DomainKeys Identified Mail (DKIM) is a specification for cryptographically signing e-mail messages, permitting a signing domain to claim responsibility for a message in the mail stream.
- Message recipients (or agents acting in their behalf) can verify the signature by querying the signer's domain directly to retrieve the appropriate public key and thereby can confirm that the message was attested to by a party in possession of the private key for the signing domain.
- DKIM has been widely adopted by a range of e-mail providers, including corporations, government agencies, gmail, yahoo, and many Internet Service Providers (ISPs).

4.10.1 Internet Mail Architecture

- **Message User Agent (MUA):** Operates on behalf of user actors and user applications. It is their representative within the e-mail service. Typically, this function is housed in the user's computer and is referred to as a client e-mail program or a local network e-mail server. The author MUA formats a message and performs initial submission into the MHS via a MSA. The recipient MUA processes received mail for storage and/or display to the recipient user.
- **Mail Submission Agent (MSA):** Accepts the message submitted by an MUA and enforces the policies of the hosting domain and the requirements of Internet standards. This function may be located together with the MUA or as a separate functional model. In the latter case, the Simple Mail Transfer Protocol (SMTP) is used between the MUA and the MSA.
- **Message Transfer Agent (MTA):** Relays mail for one application-level hop. It is like a packet switch or IP router in that its job is to make routing assessments and to move the message closer to the recipients. Relaying is performed by a sequence of MTAs until the

message reaches a destination MDA. An MTA also adds trace information to the message header. SMTP is used between MTAs and between an MTA and an MSA or MDA.

- **Mail Delivery Agent (MDA):** Responsible for transferring the message from the MHS to the MS Fig 4.9.
- **Message Store (MS):** An MUA can employ a long-term MS. An MS can be located on a remote server or on the same machine as the MUA. Typically, an MUA retrieves messages from a remote server using POP (Post Office Protocol) or IMAP (Internet Message Access Protocol).

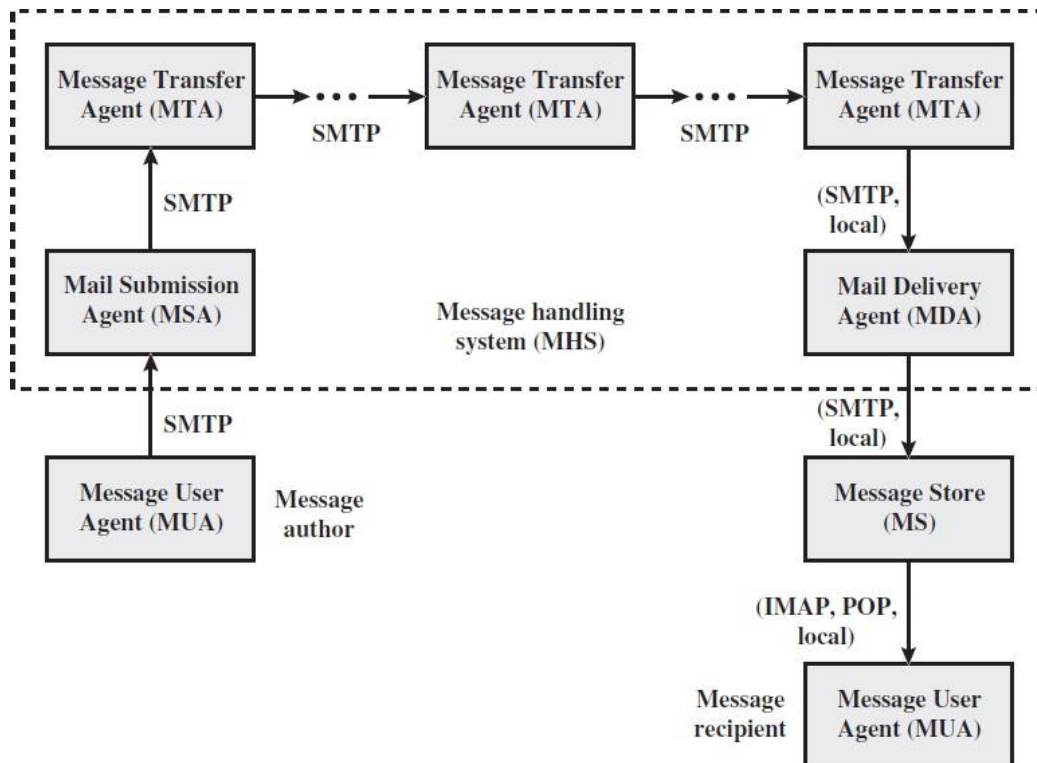


Fig 4.9: Internet Mail Architecture

4.11 E-mail Threats

Characteristics

1. At the low end are attackers who simply want to send e-mail that a recipient does not want to receive. The attacker can use one of a number of commercially available tools that allow the sender to falsify the origin address of messages. This makes it difficult for the receiver to filter spam on the basis of originating address or domain.

2. At the next level are professional senders of bulk spam mail. These attackers often operate as commercial enterprises and send messages on behalf of third parties. They employ more comprehensive tools for attack, including Mail Transfer Agents (MTAs) and registered domains and networks of compromised computers (zombies) to send messages and (in some cases) to harvest addresses to which to send.
3. The most sophisticated and financially motivated senders of messages are those who stand to receive substantial financial benefit, such as from an e-mail-based fraud scheme. These attackers can be expected to employ all of the above mechanisms and additionally may attack the Internet infrastructure itself, including DNS cache-poisoning attacks and IP routing attacks.

4.11.2 Capabilities

1. Submit messages to MTAs and Message Submission Agents (MSAs) at multiple locations in the Internet.
2. Construct arbitrary Message Header fields, including those claiming to be mailing lists, resenders, and other mail agents.
3. Sign messages on behalf of domains under their control.
4. Generate substantial numbers of either unsigned or apparently signed messages that might be used to attempt a denial-of-service attack.
5. Resend messages that may have been previously signed by the domain.
6. Transmit messages using any envelope information desired.
7. Act as an authorized submitter for messages from a compromised computer.
8. Manipulation of IP routing. This could be used to submit messages from specific
9. IP addresses or difficult-to-trace addresses, or to cause diversion of messages to a specific domain.
10. Limited influence over portions of DNS using mechanisms such as cache poisoning.
11. This might be used to influence message routing or to falsify advertisements of DNS-based keys or signing practices.
12. Access to significant computing resources, for example, through the conscription of worm-infected “zombie” computers. This could allow the “bad actor” to perform various types of brute-force attacks.
13. Ability to eavesdrop on existing traffic, perhaps from a wireless network.

4.12.3 Location

DKIM focuses primarily on attackers located outside of the administrative units of the claimed originator and the recipient. These administrative units frequently correspond to the protected portions of the network adjacent to the originator and recipient.

4.12.4 DKIM Strategy

1. S/MIME depends on both the sending and receiving users employing S/MIME. For almost all users, the bulk of incoming mail does not use S/MIME, and the bulk of the mail the user wants to send is to recipients not using S/MIME Fig 4.10.
2. S/MIME signs only the message content. Thus, RFC 5322 header information concerning origin can be compromised.
3. DKIM is not implemented in client programs (MUAs) and is therefore transparent to the user; the user need take no action.
4. DKIM applies to all mail from cooperating domains.
5. DKIM allows good senders to prove that they did send a particular message and to prevent forgers from masquerading as good senders.

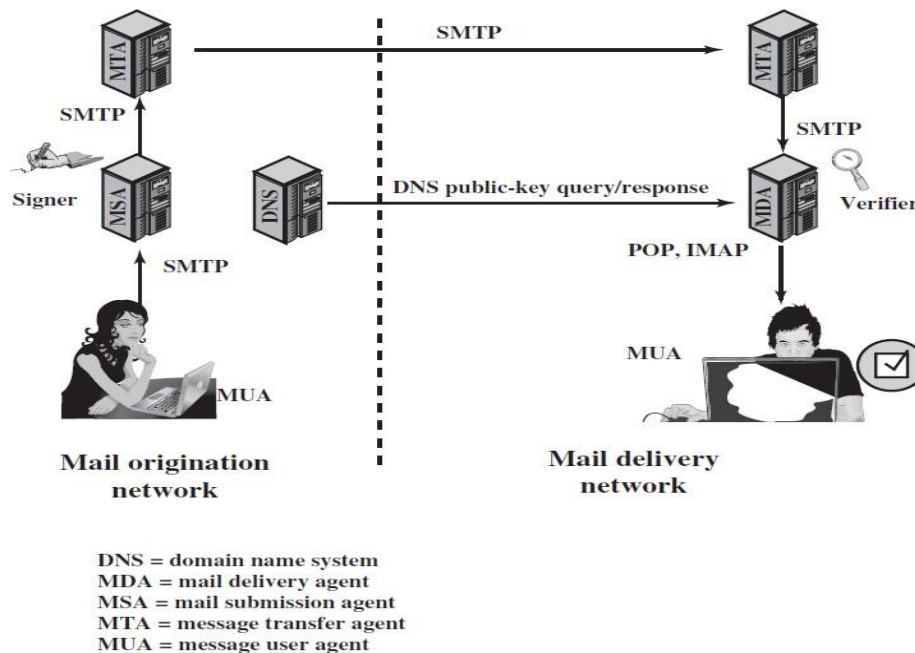


Fig 4.10: DKIM Strategy

4.12.5 DKIM Functional Flow

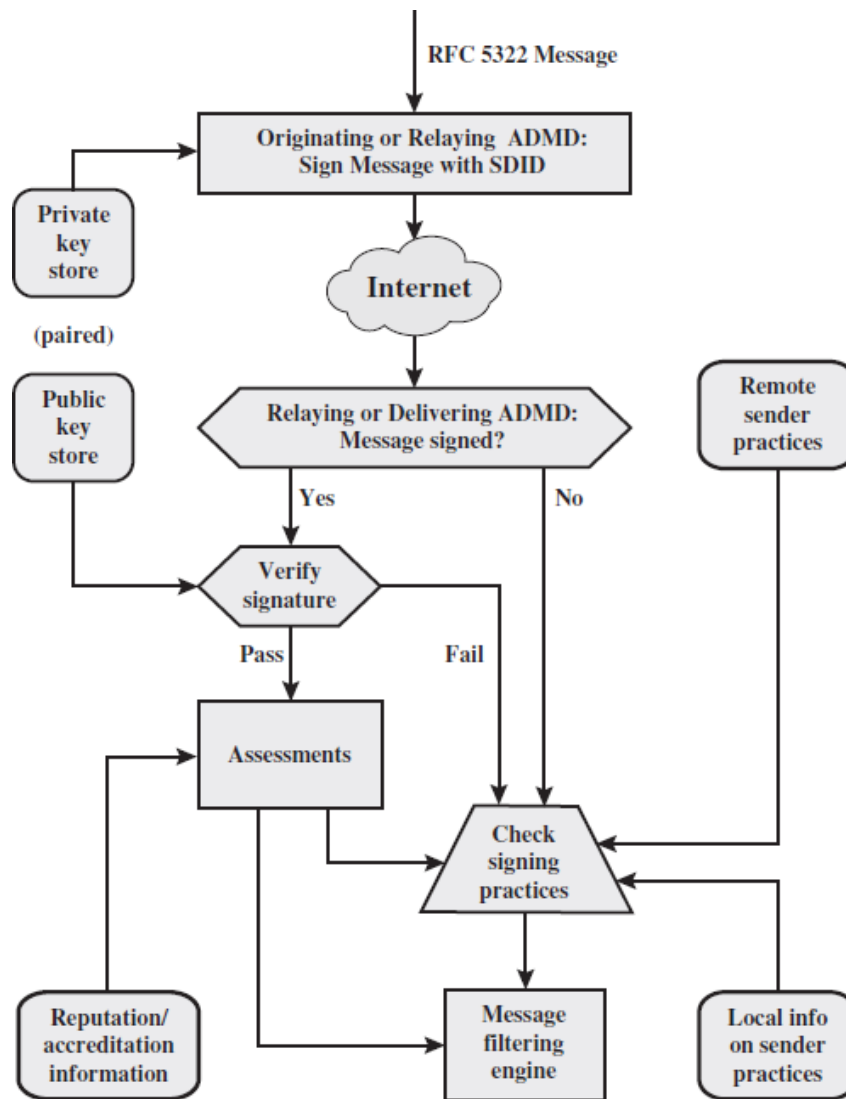


Fig 4.11: DKIM Functional Flow

Basic message processing is divided between a signing Administrative Management Domain (ADMD) and a verifying ADMD. At its simplest, this is between the originating ADMD and the delivering ADMD, but it can involve other ADMDs in the handling path Fig 4.11.

Signing is performed by an authorized module within the signing ADMD and uses private information from a Key Store. Within the originating ADMD, this might be performed by the MUA, MSA, or an MTA.

Verifying is performed by an authorized module within the verifying ADMD. Within a delivering ADMD, verifying might be performed by an MTA, MDA, or MUA. The module verifies the signature or determines whether a particular signature was required. Verifying the signature uses public information from the Key Store. If the signature passes, reputation information is used to assess the signer and that information is passed to the message filtering system.

If the signature fails or there is no signature using the author's domain, information about signing practices related to the author can be retrieved remotely and/or locally, and that information is passed to the message filtering system. For example, if the sender (e.g., Gmail) uses DKIM but no DKIM signature is present, then the message may be considered fraudulent.